

Execution Flow of xv6 Shell

Question 3 – OS HW-1

M. Zayed Nauman

Syscalls the shell primarily uses:

The shell relies on core xv6 syscalls:

- fork, wait, exec for process creation & management.
- open, close, dup for file/pipe management.
- Chdir for cd

1. Program Start: main()

```
int main(void)
{
    static char buf[100];
    int fd;

    //Ensure 3 file descriptors (stdin=0, stdout=1, stderr=2) are open.
    while((fd = open("console", O_RDWR)) >= 0){
        if(fd >= 3){
            close(fd);
            break;
        }
    }

    //Main loop: read and run commands
    while(getcmd(buf, sizeof(buf)) >= 0){
        ...
    }
    exit(0);
}
```

Shell ensures that stdin, stdout, stderr point to console. Then enters an infinite loop to prompt the user, read input line, then parse and execute it.

2. Reading Input: getcmd()

```
int getcmd(char *buf, int nbuf)
{
    write(2, "$ ", 2);
    memset(buf, 0, nbuf);
    gets(buf, nbuf);
    if(buf[0] == 0)
        return -1;
    return 0;
}
```

Prints \$ prompt to take user command, then takes it into the buffer.

3. Main Loop Processing

Back in `main()` after `getcmd()`:

```
char *cmd = buf;
while (*cmd == ' ' || *cmd == '\t')
    cmd++;
if (*cmd == '\n')
    continue;

if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '){
    // Built-in cd
    ...
} else {
    if (fork1() == 0)
        runcmd(parsecmd(cmd)); //child executes command
    wait(0); //parent waits
}
• Case
```

If user types `cd <dir>` then it's handled directly in parent. Else parent forks child `fork1()`, child parses the command string using `parsecmd()` then execute `runcmd()`, parent waits for child to finish by calling `wait()`.

4. Parsing Commands: `parsecmd()`

```
struct cmd* parsecmd(char *s)
{
    char *es = s + strlen(s);
    struct cmd *cmd = parseline(&s, es);
    ...
    return nulterminate(cmd);
}
```

The parser does the job of breaking the raw string into structured commands for executing

`parseline()` → `parsepipe()` → `parseexec()` → `parseredirs()` → `nulterminate()`

5. Executing Commands: `runcmd()`

Using cases EXEC, REDIR, LIST, PIPE, BACK, the **engine** that interprets the parsed command are in an work-able hierarchy and actually executes:

EXEC: allows to run the program by making the current command wait

PIPE: Connect output of one command to input of another through a path called a pipe.

LIST: Run commands sequentially

BACK: Run a command in background without making it wait